AI ENGINEERING
AI-Powered products

MÓDULO 4: Arquitectura RAG

Sesión 10: Técnicas avanzadas de recuperación

Mentor: Julián Fernández
LIDR.co, training the Tech Leaders' of the future

L1DR

Hoy: de un índice a un pipeline

Decidir dónde buscar, cómo preguntar, qué excluir y cómo desempatar

S8	S9	**S10 · HOY**	S11
pgvector	RAG y estimate	recuperación avanzada	generación y calidad

Bloque 1 · 15 min
Lo que traéis: RRF, idioma del tsvector y reranking

Bloque 2 · 90 min
Multi-índice y routing · expansión y descomposición · filtros, decaimiento y arnés

Bloque 3 · 25 min
Del retrieval al producto: horas por tarea con humano en el lazo

Al salir de hoy sabréis…

- Elegir y ordenar las etapas de un pipeline de recuperación.
- Mandar cada pregunta a su colección.
- Medir cada técnica antes de dejarla encendida.
- Anclar las horas de un presupuesto en datos históricos.

Regla de oro: cada etapa entra una lista de chunks y sale una lista de chunks. Un toggle la enciende o la apaga sin tocar código.

Lo que traéis del ejercicio previo

Los tres cimientos que reutiliza el resto de la sesión

1.1 · Búsqueda híbrida + RRF

Rama vectorial + rama léxica (`tsvector`) fusionadas por posición. Vuelve en la expansión de consultas.

1.2 · Idioma del tsvector

Columna y consulta con la **misma** configuración, la del idioma del corpus. Si no, el índice se ignora en silencio.

1.3 · Reranking

Cross-encoder sobre los 50 mejores. **Hoy no se reprograma**: se calienta, se compone como etapa y se mide.

Y el arnés

`scripts/eval_retrieval_s10.py` + golden set: cada configuración es un dato, no una rama de código. Hoy añadimos filas a vuestra tabla.

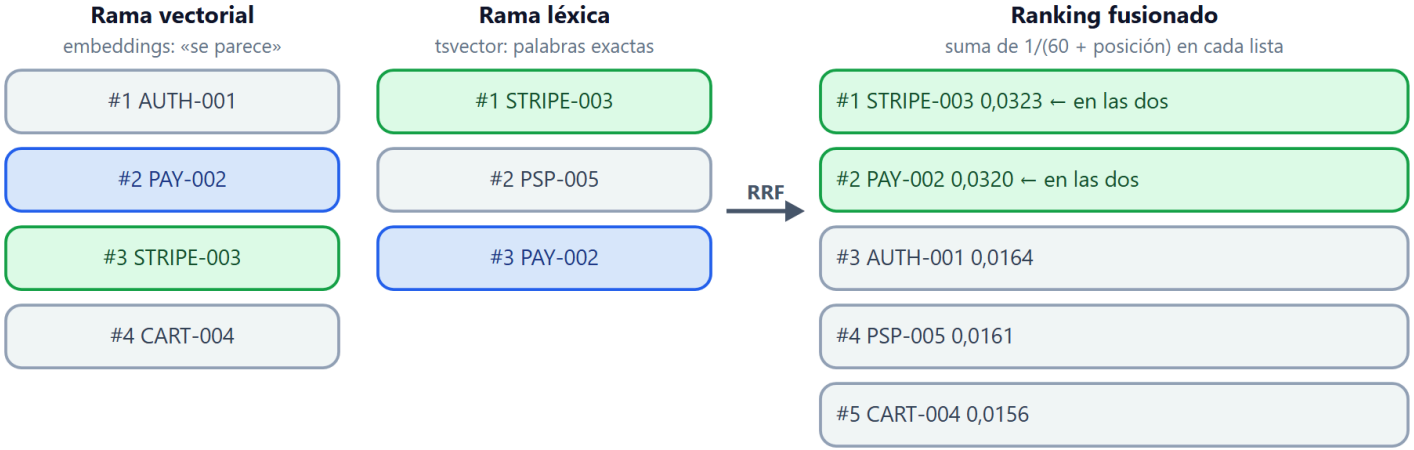
▶ **En el código** scripts/eval_retrieval_s10.py · `CONFIGS` (l. 93) · app/generation/rag/retrieval/fusion.py · reranker.py

1.1 · Calibración de RRF

app/generation/rag/retrieval/fusion.py · RRF_K = 60

RRF: fusiona por posición, no por puntuación

score(d) = Σ 1 / (k + rank) · k = 60 · consulta «integration with Stripe Connect» (ilustrativo)



AUTH-001 era el #1 vectorial y cae al #3: gana quien va bien en las DOS ramas. k grande aplana; k pequeño deja dominar a un #1.

Rankings, no scores: la distancia del vector y ts_rank_cd no son comparables entre sí

60 es el valor del paper de Cormack et al. y nuestro default: no se optimiza sin evidencia.

▶ **En el código** app/generation/rag/retrieval/fusion.py · `reciprocal_rank_fusion()` (l. 25) · app/config.py · `RRF_K` (l. 150)

1.2 · El idioma del tsvector

La configuración sigue al idioma del corpus, no al del enunciado

Las dos referencias deben coincidir

```
# store/models.py — columna generada
content_tsv = Computed(
    "to_tsvector('english', content)",
    persisted=True)

# store/repository.py — la consulta
plainto_tsquery("english", query_text)
```

La trampa

El enunciado dice "corpus en español", pero el dataset está **en inglés**. Poner `spanish` sobre texto inglés rompe el stemming y el híbrido degrada a solo vectorial **sin avisar**.

Demo · comprobarlo en psql

```
psql> SELECT to_tsvector('english',
          'Stripe Connect onboarding')
       @@ plainto_tsquery('english',
          'stripe onboarding') AS matches;
 matches
---------
 t
```

Qué aporta al estimador

La rama léxica es la que encuentra los identificadores exactos (`Stripe`, `SAP`, `BUD-2024-014`) que el embedding diluye en "algo de pagos".

▶ **En el código** app/generation/rag/store/models.py · `FTS_REGCONFIG` (l. 54), `content_tsv` (l. 100) · repository.py · `search_lexical()` (l. 175)

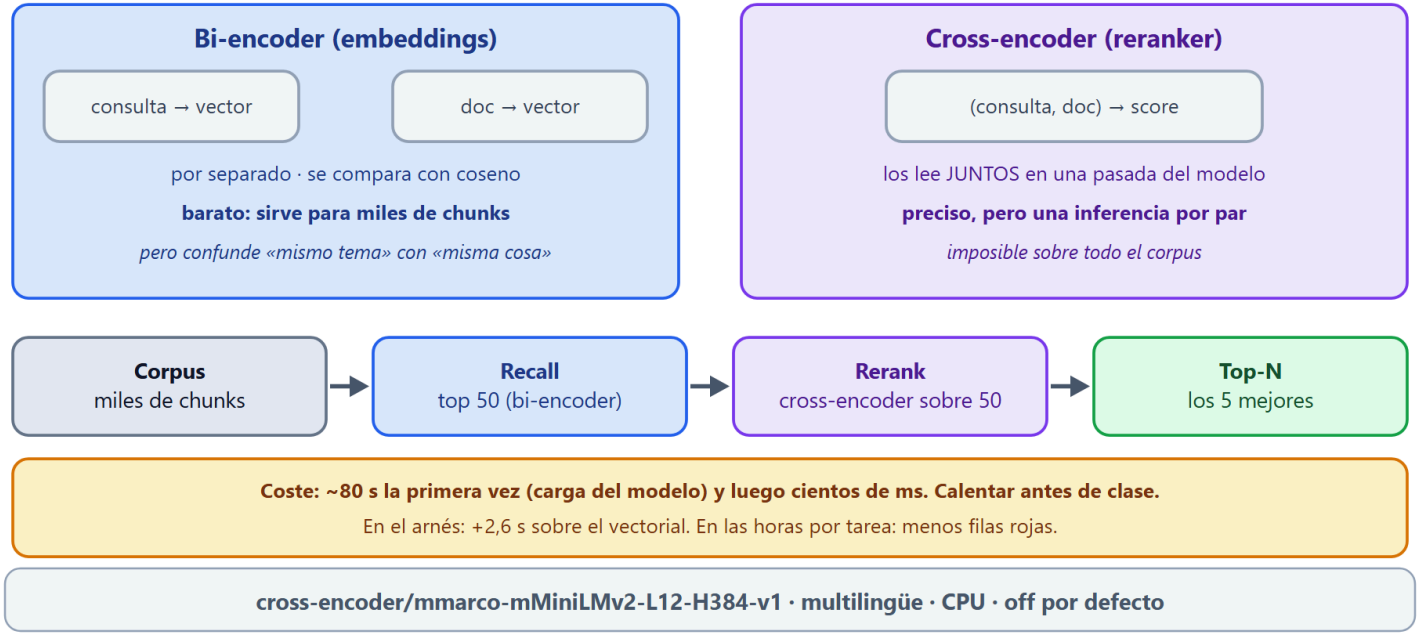
L1DR

1.3 · Reranking: cross-encoder y latencia en frío

app/generation/rag/retrieval/reranker.py

Recall-then-rerank: bi-encoder para buscar, cross-encoder para ordenar

El reranking del ejercicio previo: hoy no se reprograma, se compone y se mide



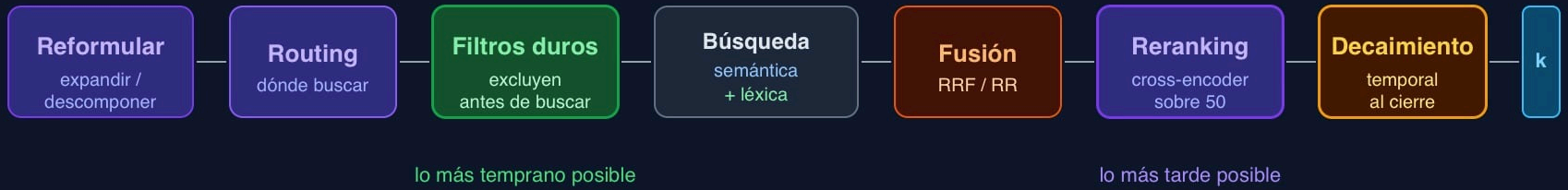
Antes de clase: `python -m app.generation.rag.retrieval.verify_reranker`

▶ **En el código** app/generation/rag/retrieval/reranker.py · `score()` (l. 77), `rerank_with_scores()` (l. 97) · advanced_pipeline.py etapa 6 (l. 263)

▶ **Código** app/generation/rag/retrieval/advanced_pipeline.py · `advanced_retrieve()` l. 160

El orden del pipeline es el mensaje

Lo barato y excluyente, **al principio.**
Lo caro y fino, **al final.**
Lo blando, **al cierre.**



Cada etapa es **componible y conmutable** : entra una lista de chunks, sale una lista de chunks.
El pipeline completo es el camino máximo, no el obligatorio — y por eso cada pieza se mide aislada.

▶ **Código** app/generation/rag/retrieval/router.py · `route()` l. 73

Routing en cascada por coste creciente

El mejor router es no tener router. Se sube de nivel solo cuando el anterior no decide.

0 — **Explícito — el contrato de la API** — coste 0
el llamante nombra la colección. No se clasifica nada.

no resuelto

1 — **Reglas deterministas** — coste 0
patrones de vocabulario: "qué dijo el cliente" → transcript

no resuelto

2 — **Clasificador LLM** — +1 LLM
salida estructurada: 1–3 destinos + reason. La duda = varios destinos.

no decide

3 — **Fallback — buscar en todas**
en paralelo. Degradación elegante: nunca peor que el índice único.

Tres colecciones

budget_chunks
cuánto costó
esquema e índice propios

transcript_chunks
qué pidió el cliente
esquema e índice propios

technical_doc_chunks
cómo se implementa
esquema e índice propios

la procedencia viaja en cada chunk

Routing en directo: los tres primeros niveles

POST /v1/retrieval/advanced-search

Nivel 1 · reglas (vocabulario de reunión, sin LLM)

```
«what did the client say in the meeting about Stripe checkout»
→ level: rules   targets: [transcript]   technique: expand   cardinality: {transcript: 11}
```

Nivel 2 · clasificador LLM

```
«OAuth 2.0 PKCE flow and PSD2 strong customer authentication reference»
→ level: classifier   targets: [technical_doc]
  reason: "...technical topics best addressed by internal technical documentation."
```

Nivel 0 · explícito

```
«telemedicine appointment scheduling»  +  "collections": ["budget"]
→ level: explicit     targets: [budget]           (el contrato manda, no se clasifica)
```

▶ **En el código** **Interfaz:** Búsqueda → «Routing · reglas / clasificador / explícito» · app/generation/rag/retrieval/router.py · `route()` (l. 73) · collections.py · `match_rules()` (l. 195) · app/api/routers/retrieval_advanced.py (l. 92)

▶ **Código** app/generation/rag/retrieval/query_transform.py · l. 82–142

Expansión vs descomposición

¿Una cosa dicha de muchas maneras, o muchas cosas dichas a la vez?

Expansión (multi-query)

N formulaciones de la misma intención

combate la lotería del vocabulario

"que los comerciales vean sus números en el móvil"

↓

"dashboard de KPIs comerciales, versión móvil"

"informes de ventas accesibles desde smartphone"

Fusión por consenso

RRF: lo que todas las variantes respetan, flota

Descomposición

sub-consultas independientes por tema

combate el embedding promedio multi-tema

"login, cobros con Stripe, recomendaciones de catálogo..." (transcripción)

↓

login / RBAC | cobros Stripe | catálogo

Fusión por cobertura

round-robin: cada tema con representación garantizada

Si falla el transformador, la recuperación degrada a consulta directa. Nunca depende de él.

Descomposición en directo

Una transcripción multi-tema se parte por temas

Demo · ejecución real

```
query: "we need OAuth and a transaction ledger,
  plus a Stripe checkout, and also personalized
  recommendations for the product catalog"

→ technique: decompose      (4 sub-consultas)
  · OAuth
  · transaction ledger
  · Stripe checkout
  · personalized recommendations

  p. ej. «How to integrate Stripe checkout
          into an e-commerce website?»
```

Consulta corta, un tema

```
query: "Stripe checkout"

→ technique: direct
  (sin LLM)
```

Sin LLM cuando no hace falta

`choose_technique()` decide por longitud y conectores. Si el LLM falla, degrada a consulta directa.

Qué aporta al estimador

La entrada real es una transcripción que mezcla login, cobros y catálogo. Un solo embedding queda "cerca de todo, lejos de nada". Descomponiendo, **cada módulo del desglose tiene su evidencia**, no solo el dominante.

▶ **En el código** **Interfaz:** Búsqueda → «Descomposición» y «Consulta directa» · app/generation/rag/retrieval/query_transform.py · `choose_technique()` (l. 82) · fusion.py · `round_robin_merge()` (l. 63) · advanced_pipeline.py · `_fuse_collection()` (l. 134)

▶ **Código** app/generation/rag/retrieval/collections.py l. 106 · temporal.py l. 33

Metadatos: filtro duro vs ponderación blanda

Filtro duro

excluye ANTES de buscar · empotrado en el WHERE

"clínica con presupuestos de healthcare 2024,
no con un ecommerce de 2019"

Trampa HNSW: el post-filtrado puede vaciar en silencio
→ **loguear SIEMPRE la cardinalidad tras el filtro**

Ponderación blanda

reordena finalistas AL CIERRE · decaimiento temporal

```
weight = 0.5 ** (age_days / half_life)
```

semivida 900 días: prefiere lo reciente
sin descartar lo antiguo válido (solo desempata)

El orden no es negociable

Fusión
junta candidatos

—

Reranking
fija el orden por relevancia

—

Decaimiento
solo atenúa finalistas

¿Por qué sigmoid antes del decaimiento?

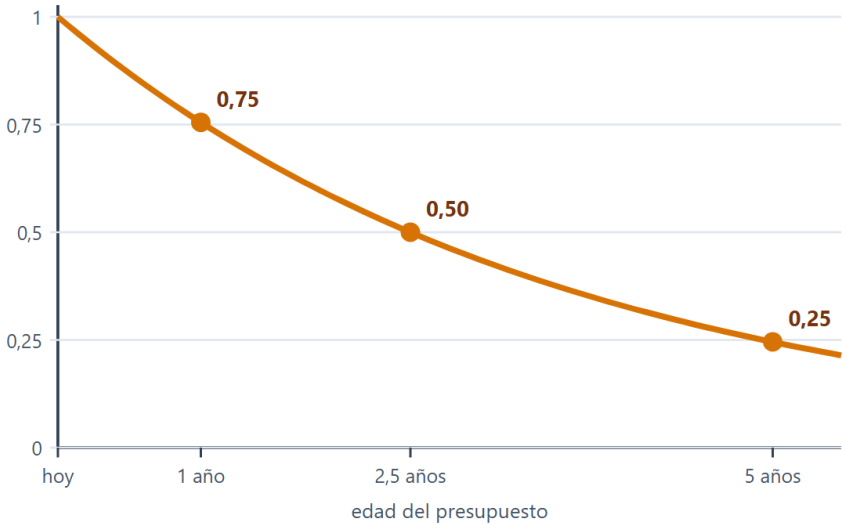
El logit del cross-encoder puede ser negativo. Un multiplicador 0..1 sobre un negativo invierte el ranking.
Sigmoid lleva el logit a (0,1) para que el decaimiento solo atenúe, nunca dé la vuelta al orden.

Decaimiento temporal

app/generation/rag/retrieval/temporal.py

Decaimiento temporal: preferir lo reciente sin descartar lo antiguo

weight = 0,5 ^ (edad_días / 900) · semivida de 900 días (≈ 2,5 años)



Por qué al cierre

Es ponderación BLANDA: solo desempata entre finalistas ya relevantes. Antes del rerank penalizaría lo que no sabes si sirve.

Por qué sigmoid antes

El logit del cross-encoder puede ser negativo: × 0,5 lo «sube». sigmoid lo lleva a (0, 1) y el decaimiento solo atenúa.

Off por defecto (TEMPORAL_DECAY_ENABLED=false): encenderlo solo con evidencia

▶ **En el código** app/generation/rag/retrieval/temporal.py · `decay_weight()` (l. 33) · advanced_pipeline.py · `_sigmoid()` (l. 128) y decaimiento (l. 279)

L1DR

Filtro duro en directo: vigilar la cardinalidad

min_date excluye antes de buscar

Demo · ejecución real

```
«telemedicine scheduling and video consultations»

sin filtro                          → budget: 1603
"min_date": "2024-01-01"            → budget:  748
+ "sectors": ["healthcare"]         → budget:   84
```

La trampa del post-filtrado

Con HNSW, filtrar después puede vaciar el resultado **en silencio**. Por eso se loguea siempre la cardinalidad tras el filtro.

Lo que nos dice la cardinalidad

Los 1603 no son 60 presupuestos: en la misma tabla viven también las **1543 tareas históricas** del Bloque 3. Sin mirar la cardinalidad no lo sabrías. Y si una demo da `low_confidence: true` con cardinalidad 0, es el filtro vaciando la colección: se relaja y se mira el log.

Para el estimador: estimar una clínica con presupuestos de healthcare de 2024, no con un e-commerce de 2019.

▶ **En el código** **Interfaz:** Búsqueda → «Filtro por fecha» · Corpus → ver las 1543 tareas · app/generation/rag/retrieval/collections.py · `hard_filter_clauses()` (l. 106) · store/repository.py · `search_filtered()` (l. 107) · log cardinalidad (advanced_pipeline.py l. 251)

El arnés: medir antes de encender

Un banco de pruebas que pasa las mismas preguntas por cada configuración

1 · Golden set

8 preguntas con la respuesta correcta ya marcada: qué presupuestos, transcripciones o documentos deberían salir.

2 · Configuraciones

Cada fila (A…H) es un juego de toggles: vector, híbrido, rerank, routing… Son **datos**, no ramas de código.

3 · Se mide

Precision@5: de los 5 primeros resultados, cuántos son correctos. **Latencia**: cuánto tarda.

Por qué importa

Cada técnica tiene un coste (latencia, llamadas al LLM). El arnés dice si **esa** técnica mejora **tus** preguntas antes de dejarla encendida en producción.

▶ **En el código** scripts/eval_retrieval_s10.py · `CONFIGS` (l. 93) · evals/golden_retrieval.json · `docker compose -p aieng-s10live exec estimator python scripts/eval_retrieval_s10.py`

▶ **Código** scripts/eval_retrieval_s10.py · CONFIGS l. 93

El arnés no miente: una técnica no es universalmente buena

Config	Stages	Precision@5	Latency
A	Vector	0.75	197 ms
B	Hybrid	0.75	204 ms
D	Hybrid + Rerank	0.75	3273 ms
E	Hybrid + Rerank + Temporal	0.75	2952 ms
F	Multi-index routing	0.25	1593 ms
G	Query transform	0.75	9931 ms
H	Full pipeline	0.25	6585 ms

Lectura honesta

El routing MEJORA las queries cross-colección (Q7: 0.60→0.80; Q8: 0.00→0.40)...

...y HUNDE las descripciones de proyecto (Q1–Q5 → 0.00): no citan "budget/cost", el clasificador las manda a transcript/doc y pierde los presupuestos.

Moraleja: el routing ayuda cuando la query señala su tipo. Por eso importan el nivel-0 y un buen golden set.

Nuestra ejecución del arnés

El mismo arnés sobre nuestro corpus (con las 1543 tareas en la tabla de presupuestos)

Config	Etapas	P@5	Latencia
A	Vector	**0,48**	229 ms
B	Hybrid	**0,48**	225 ms
C	Vector + Rerank	**0,62**	1142 ms
D	Hybrid + Rerank	**0,62**	1131 ms
E	Hybrid + Rerank + Temporal	**0,48**	1148 ms
F	Multi-index routing	**0,25**	1451 ms
G	Query transform	**0,62**	3002 ms
H	Full pipeline	**0,25**	1961 ms

El reranking sí paga aquí

Q3 y Q4 pasan de 0,20 a 0,60. Con ruido en el índice, el cross-encoder separa lo parecido de lo correcto.

El decaimiento rompe una consulta

Q5 cae de 0,80 a 0,00: la buena respuesta era antigua. Por eso va apagado por defecto.

El routing, igual que en LIDR

Gana en las consultas que cruzan colecciones (Q8: 0 → 0,40) y hunde las de presupuestos (Q1–Q5 → 0).

▶ **En el código** `docker compose -p aieng-s10live exec estimator python scripts/eval_retrieval_s10.py` · la matriz por consulta sale debajo de la tabla

▶ **Código** app/generation/rag/task_hours.py · `_consensus()` l. 63

Del retrieval al producto: separar estructura de horas

El LLM descompone bien y pone números mal. Separamos las dos decisiones, con el humano en el lazo.

Generación
SOLO estructura
módulos → tareas
sin horas

Revisión #1
humano edita
módulos / tareas
☛

Horas por tarea
búsqueda vectorial
consenso + fiabilidad
rojo si no hay análogo

Revisión #2
horas + tarifas
confirmar
☛

estimación
confirmada

Horas = consenso de vecinos

$$\text{horas} = \Sigma\, w_i \cdot h_i \;/\; \Sigma\, w_i \;,\; w_i = 1/(\varepsilon + d_i)$$

fiabilidad = similitud_ponderada · (1 – dispersión)

alta solo si los vecinos están cerca Y de acuerdo

sin vecino bajo umbral → sin horas, rojo, lo decide una persona

El reranking se paga aquí

La calidad del consenso depende de QUÉ vecinos entran.

Con rerank, los k vecinos son los que el cross-encoder juzga realmente parecidos.

→ menos rojos, número más estable

Horas por tarea: análogo o fila roja

POST /v1/estimate/tasks/hours · consenso de los vecinos

Demo · ejecución real

```
"Payment gateway integration"
  "Card payments via a PSP with 3-D Secure"
→ estimated_hours: 42
  reliability:     0.505    dispersion: 0.278
  has_match:       true     neighbors: 5
    TASK-2024-0003 · 54 h · distance 0.29 ...

"Quantum teleporter"  ("nonexistent")
→ estimated_hours: null
  has_match:       false      ← fila roja
```

El número es trazable

Cada hora viene con sus `neighbors`: los proyectos históricos de los que sale. Si un número es raro, se mira de dónde viene. **No es una caja negra.**

Ajustes en caliente

`TASK_HOURS_TOP_K` (vecinos del consenso) y `TASK_HOURS_DISTANCE_THRESHOLD` (el umbral del rojo) se cambian sin reiniciar con `PUT /api/v1/config/retrieval` (Swagger: localhost:8000/docs). Con el reranking encendido, menos rojos.

▶ **En el código** app/generation/rag/task_hours.py · `_consensus()` (l. 63), `estimate_one()` (l. 99) · app/api/routers/estimate_tasks.py (l. 40)

El wizard de estimación, paso a paso

Rails · Rag::EstimationRunsController

1 · Transcripción
el brief del cliente

2 · Reformulación
brief limpio

3 · Estructura
el LLM propone módulos y tareas, sin horas

4 · Revisión #1
un humano edita la estructura

5 · Horas
búsqueda vectorial por tarea, rojo si no hay análogo

6 · Revisión #2
horas y tarifas €/h · confirmar

El `POST /v1/estimate/from-transcript` de la S9 sigue intacto: el LLM infiere las horas frente a horas derivadas de evidencia.

▶ **En el código** Interfaz: **localhost:3000/rag/estimation_runs/new** · estimator-web/app/controllers/rag/estimation_runs_controller.rb · prompt_builder.py · `build_system_prompt(include_hours=False)` (l. 14)

El cuadro de mando: cada técnica, cuándo y a qué precio

Encenderla solo con evidencia del arnés

Técnica	Qué resuelve	Coste	Cuándo
Híbrido + RRF	identificadores exactos	~0 ms	casi siempre
Reranking	orden por relevancia real	+2,6 s	precisión importa más que latencia
Routing	mandar a la colección correcta	+1 LLM pequeño	la consulta señala su tipo
Expansión	vocabulario distinto	+1 LLM	consulta corta y ambigua
Descomposición	varios temas en una	+1 LLM	transcripciones largas
Filtros duros	comparar solo lo comparable	baja el coste	hay metadatos fiables
Decaimiento	preferir lo reciente	~0 ms	las tarifas cambian con el tiempo

Una técnica no es universalmente buena. El routing mejora las consultas que cruzan colecciones y hunde las que no dicen su tipo. Mide cada etapa sobre tu caso de uso.

▶ **En el código** app/config.py (l. 141–184) · toggles en caliente: `GET/PUT /api/v1/config/retrieval` (Swagger: localhost:8000/docs)

Lo que cierras hoy · lo que abre la S11

Hoy

- Tres colecciones y un router en cascada.
- Expansión y descomposición de consultas.
- Filtros duros y decaimiento temporal.
- Un pipeline de etapas conmutables y medibles.
- Horas por tarea ancladas en datos, con humano en el lazo.

Sesión 11 · RAG avanzado

- Generación con el contexto recuperado.
- Calidad: cómo medir si la respuesta es buena.
- El router acotado de hoy es el precursor del *handover* agéntico.

Todo el código de referencia está en la rama `session_10_live`.

✋Tiempo para el feedback con ROTI

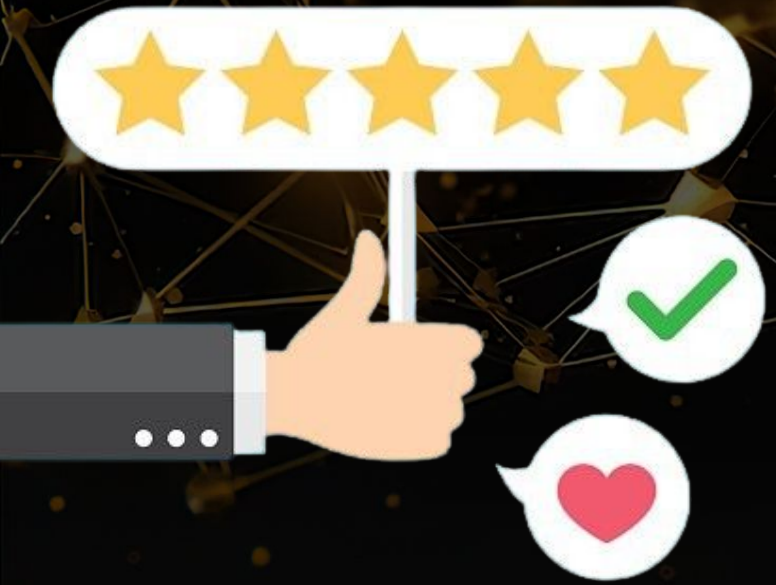
¿Cuánto valor te ha agregado esta sesión?



Absolutamente nada	Poco	Suficiente	Bastante	Más de lo que esperaba

LIDR

ROTI



¡GRACIAS!

LIDR.co, AI-Powered Careers

@lidr_ai

#followtheLIDR